PROJECT AT A GLANCE

Memorize, Retrieve, or Query?

LLMs answering questions about a software project's history

A five-page guide to the problem, actual data, system, evaluation, and current progress.

1. The problem and why we are studying it

Software projects such as Firefox accumulate years of bug reports, developer comments, status changes, and links between related issues. A person asking *"How long did this bug take to fix, and why?"* may have to search several records and calculate a timeline manually. A chatbot can make this easier, but a convincing answer may still be wrong: it might invent a date, confuse *RESOLVED* with *FIXED*, or infer a reason not stated in any comment.

Our goal is not to build a production help desk or train a new LLM. We are developing a **local research prototype** to compare how different ways of accessing project knowledge affect **answer correctness, evidence support, and the ability to recognize missing information**. A small Streamlit interface will demonstrate the methods; the central deliverable is the controlled experiment.

Research questions

RQ1	Which approach answers each kind of question most accurately: model memory, text retrieval, hybrid retrieval, or a structured database query?
RQ2	Which approach makes fewer unsupported claims, cites genuine evidence, and abstains when the answer is unavailable?
RQ3	What degrades when bug information becomes outdated, and how much does a controlled data refresh recover?

Intended users and the application's requirements

The immediate users are project teammates and course evaluators; possible future users are maintainers, researchers, and newcomers to large software projects. A user enters a natural-language question, chooses an answering method, and sees an answer, source references, and response time.

The five priorities are: **(1)** support common bug-history questions; **(2)** provide evidence or say that it is missing; **(3)** compare methods on identical questions; **(4)** reproduce results from a frozen dataset and configuration; and **(5)** test data refreshes. Non-functional constraints include local execution, privacy-aware text handling, no paid API requirement, and straightforward Git-based reproducibility.

> **Scientific distinction.** "Time to first FIXED" means elapsed calendar time from filing to a verified FIXED transition. It does *not* measure engineer-hours, active work time, or necessarily the date a patch was merged.

What is new relative to the original proposal?

The proposal began as a ChatGPT-like interface over Firefox data with possible RAG and graph comparisons. The study is now narrower and measurable: four named baselines, six question families, verified reference answers, explicit missing-history rules, and a controlled freshness experiment. Knowledge-graph retrieval is optional, not required for the core study.

2. The data: what it is, what it means, and what we learned

We use **BugsRepo** (Acharya and Ginde, EASE 2025), collected from Mozilla's Bugzilla platform, and selectively enrich it with public Bugzilla REST history records. The paper describes a curated collection of **119,585** resolved/closed reports from more than 50 Mozilla projects, through October 2024, plus contributor and quality-filtered subsets. Our downloaded *raw metadata export* is a **different, broader population**, including open and reopened bugs. These numbers must not be combined or treated as interchangeable.

Verified source-file audit (before canonical deduplication)

Source	Rows	Distinct bug IDs	Use
`Bug_meta_data.csv`	225,196	196,633	Base structured facts: bug ID, product, component, summary, creation, status, resolution, and links.
`comments_Dataset_Part_1.csv`	500,000	50,764	Main discussion-text source; repeated joined bug metadata.
`comments_Dataset_Part_3.csv`	11,057	1,480	Additional comments; 1,479 new comment-covered bug IDs.
Part 2 comments	500,000	50,764	Byte-identical to Part 1; *excluded*.
Filtered bug reports	120,047	83,596	Optional quality tier; not pooled into base counts.

Phase 3A full-file audit. Parts 1 + 3 contain 511,057 source comment rows across 52,243 distinct bugs; those rows require additional record-level deduplication.

How the records are organized

A **bug** has a unique ID and snapshot attributes (such as `status=RESOLVED` and `resolution=FIXED`). Many **comments** can refer to that bug; each has text and a comment timestamp. Some bugs have recorded **relationships** such as `depends_on`, `blocks`, or `duplicate_of`. A fourth entity, **field-change events**, is needed to reconstruct exact lifecycle histories; our selected CSVs do not contain a verified ordered event stream.

Five findings that directly affect the experiment

1. **Not Firefox-only:** the raw metadata has 29,919 Firefox-product rows, 93,645 Core rows, 10,370 Toolkit rows, and 91,262 other Mozilla rows. We define a Firefox-product sample explicitly, rather than calling all Mozilla data Firefox.
2. **Status is not resolution:** the raw metadata includes 95,567 FIXED-resolution rows, but also DUPLICATE, INVALID, INCOMPLETE and open cases. `RESOLVED` alone does not mean fixed.
3. **Comment-selection bias:** all 511,057 selected raw comment rows have a FIXED snapshot resolution. Explanatory findings cannot be generalized to open/unresolved bugs. Bugs with no comments stay eligible for factual tasks.
4. **Duplicated and noisy exports:** Part 2 repeats Part 1, IDs repeat within files, and some source fields are malformed or ambiguous. We hash, normalize, deduplicate, and retain provenance.
5. **Missing transition history:** `last_change_time` is not the first FIXED event. We use selected Bugzilla `/history` events instead of inventing fixing dates. The exact per-field historical validity of the broader export is not established just by the paper's October 2024 coverage date.

After preprocessing. Phase 5's frozen input summary reports **196,633 canonical bugs**, **419,458 normalized comments**, and a **3,000-bug seeded sample** (seed 317). The base `field_changes` table contains zero rows; some F2 questions separately cite selected Phase 4 Bugzilla history evidence.

3. System architecture and why we chose each method

All methods use the same declared benchmark and data-access rules. We clean source files once rather than let each answering system independently collect records. This prevents data-quality differences from being mistaken for model differences.

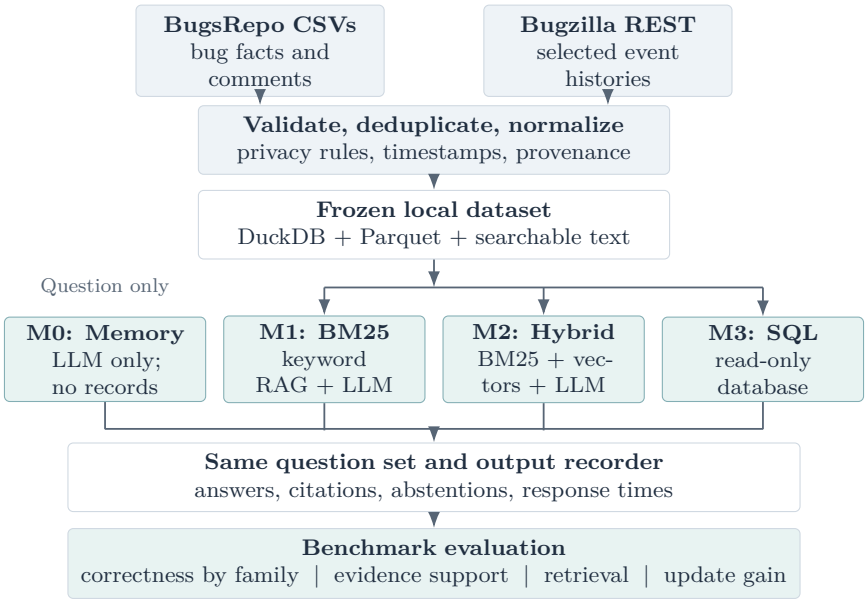


Figure 1. Research architecture (simplified). M0 receives no project records; M1–M3 access controlled evidence. The UI is a demonstration layer, while benchmark scoring drives the study.

The four required approaches

Method	How it works	Why we need it / expected weakness
M0: closed-book	Send the question to a pretrained LLM with no project context.	Measures memorization; may guess private or obscure bug-specific facts.
M1: BM25 RAG	Search for matching bug/comment terms; give retrieved passages to the LLM.	Simple, strong lexical baseline; may miss synonyms and paraphrases.
M2: hybrid RAG	Combine BM25 with local sentence embeddings and FAISS vector search; pass evidence to the same LLM.	Tests semantic retrieval's added value; similar-topic results can concern the wrong bug.
M3: SQL	Parse supported question patterns and run read-only, parameterized queries in DuckDB.	Exact counts, fields, and verified durations; not a general explanation generator.

Concrete tools and cost choices

Python for pipelines/tests; **DuckDB 1.4.4 + Parquet** for data; **BM25** for keyword search; local **Sentence Transformers + FAISS** for semantic search; an eligible free-tier **Gemini** model (or **Ollama** locally) for generation; **Streamlit** for the demo; **Matplotlib, JSON manifests, and GitHub** for evaluation and reproducibility. No model fine-tuning or paid retrieval API is necessary. The Gemini key is needed only when generative experiments begin (Phase 8); the public Bugzilla history calls generally need no key.

4. Benchmark: how we will know if an answer is correct

BugsRepo supplies bug records, *not* ready-made questions with verified answers. We therefore automatically generate questions from actual records, derive factual reference answers using validated rules or queries, and audit explanations against developer comments. Human reviewers do *not* label hundreds of thousands of bugs; they review selected question–evidence pairs.

Actual Phase 5 generation (seed 317)

Family	Example question	Candidates	Primary reference / scoring
F1	What component/status is recorded for bug X?	30	Metadata lookup; normalized exact match.
F2	How many hours to first FIXED? How often reopened?	19	Verified history events; duration error or event-count accuracy.
F3	How many eligible bugs match these filters?	30	Independent scoped counts; exact integer accuracy.
F4	Which bug IDs are recorded as dependencies?	30	Explicit relationship set; set precision, recall, F1.
F5	Why did developers say the fix was delayed?	30	Public comment evidence; human-reviewed claim support.
F6	How many engineer-hours were spent?	30	Correctly acknowledge missing evidence; abstention quality.
Total	34 development + 135 held-out test questions	**169**	Target was 180; F2 lacks 11 fully eligible questions.

There are also 6 supplemental paraphrase challenges, not 6 additional independent benchmark questions. The current benchmark has **0 historical-as-of questions**; the later freshness study needs its own validated update protocol.

Primary measurements

Metric	What it tells us
Correct-answer rate, by family	How often each approach supplies the intended verifiable answer for supported tasks.
Duration MAE and relation F1	How wrong date calculations are (hours/days), and whether linked bug sets are complete and precise.
Unsupported-claim and citation support	Whether generated statements and their citations are actually supported by visible records.
Correct / false abstention; coverage	Whether a method recognizes unavailable answers without refusing too many answerable ones.
Retrieval Recall@5/@10	Whether the search system retrieved the passages or events necessary for an answer.
Latency and resource use	Median/p95 query time, indexing overhead, local disk use, API requests and token usage.
Stale-versus-refreshed gain	Difference in correctness on identical new-data questions before and after an evidence-valid refresh.

Fairness and safeguards

All methods face the **same frozen questions**; M1 and M2 use comparable context budgets and the same generator. We select retrieval settings on the *development* partition, never tune on held-out test answers, and record model/configuration and source versions. The SQL method is scored with its coverage limitations made explicit: a free-text explanation may be *unsupported by design*, not a fabricated SQL error. We plan paired confidence intervals on headline differences and openly report unsupported question types.

Human review is essential for F5. An LLM or Codex may prepare candidate explanations, but its recommendation is not independent human verification. We retain a separate review status, discard unsupported claims, and report the verified F5 denominator.

5. What has been completed and what happens next

This status reflects the **Phase 5 generation output and sign-off queue shared on 23 October 2026**. Earlier tracked status files have not yet been updated here to reflect every later user-run command; the review CSV provided with this brief still has **all 64 required decisions pending**. No model-comparison results should be represented as completed.

Completed work and current gate

Phase	State	Outcome
0–1	Implemented	Existing `Project/` GitHub repository, Python structure, scripts, research logs, Git safety.
2–3A	Verified	Full audit/EDA: counts, product and outcome distribution, CSV overlaps, missing histories, comment selection bias.
3B	User-run complete	Normalization and frozen sample; Phase 5 observes 196,633 bugs, 419,458 normalized comments, 3,000 sampled bugs.
4	Enrichment used	Selected Bugzilla `/history` records appear in F2 gold/evidence; base `field_changes` remains empty. Exact coverage must be kept separate.
5	**Awaiting human sign-off**	Generated **169** candidates (34 dev / 135 test), 6 supplemental challenges; review queue has 175 rows, **64 required**, pending. Gold is *not yet frozen*.

Next steps, in execution order

1. **Finish Phase 5 review and freeze.** From the local simplified `signoff.html`, inspect questions and source evidence; decisions save to `signoff_queue.csv`. Reconcile any validator issues, corrections/rejections, and verify that required reviews are complete. Only then freeze the benchmark and publish its local manifest/hash.

2. **Phase 6 – SQL baseline.** Implement guarded intent parsing and parameterized DuckDB queries. Evaluate on frozen question text *without* giving the parser gold values or generator template IDs.

3. **Phase 7 – retrieval.** Build BM25 and local embedding/FAISS indexes from eligible public documents; measure evidence Recall@k independently of answer generation.

4. **Phase 8 – LLM comparisons.** Choose one free-tier-eligible Gemini model (or a pinned local Ollama alternative); generate/cache M0–M2 answers with equal evidence rules and capped requests.

5. **Phase 9 – scoring.** Measure per-family correctness, unsupported claims, citation accuracy, abstention, retrieval quality, latency, and paired uncertainty; study concrete error cases.

6. **Phase 10 – freshness experiment.** Define a defensible earlier/later evidence pair; compare stale and refreshed stores/indexes on identical update questions. Do not claim a historical cutoff without valid timestamps.

7. **Phase 11 – deliverables.** Complete the small Streamlit demo, plots, final report, limitation discussion and reproducibility instructions. A knowledge-graph extension is optional only after core methods work.

Reproducibility and operating rules

Use the existing repository `ahmadfaisal9900/Bugzilla`. The agent implements/tests small jobs, while long imports, indexing and evaluations run via executable scripts in the user's terminal. Keep raw/normalized datasets, review evidence, secrets and model outputs *outside Git*. Commit code, safe aggregate QC and decisions at each verified phase. The current sign-off folder is `outputs/benchmark/phase05/agent_review/signoff/run-65c5be3381f66811/`.

Sources/record basis: Acharya & Ginde, *BugsRepo* (EASE 2025; arXiv:2504.18806); local Phase 3A audit/QC and Phase 3B normalization notes; Phase 5 generation summary and sign-off CSV/HTML. Numerical experiment results are pending.